

LLM Chatbot Integration in Godot

Game-Based Learning Study — Game-LLM-Interactive Condition

Valentino Clerx

Fontys Master of Applied IT

2026

1. Introduction

This document provides the full analysis and design for the LLM chatbot system integrated into the leopard gecko caretaking game. The chatbot powers the Game-LLM-Interactive condition of the research study, in which players interact with an AI veterinarian character, Erwin Stevens, using open-ended natural language, as opposed to the static caresheet or fixed RPG dialogue available in the other conditions.

The purpose of this document is to record (1) the problem analysis that led to the chosen architecture, (2) the considered design alternatives and the rationale for choosing between them, and (3) the full technical design of the implemented solution, including a component diagram, data flow specification, and prompt engineering strategy.

2. Problem Analysis

2.1 Context and requirement

The research study compares four instructional conditions: a non-game caresheet, a game with an in-game caresheet, a game with hardcoded RPG dialogue, and a game with an LLM-driven conversational vet. The fourth condition requires a character that can respond to any player-typed question in natural language, scoped strictly to leopard gecko husbandry as described in the ReptiFiles caresheet. This is not a decoration, the conversational freedom of the LLM condition is the independent variable being studied.

The requirement therefore is: given a player's typed question within the game, produce a relevant, caresheet-scoped, appropriately sized response and display it in the game UI, with no blocking of the game thread, no API key exposure in client-side code, and no reliance on a custom backend server.

2.2 Constraints

Constraint	Detail
Runtime environment	Godot 4 — GDScript, no Node.js or Python runtime available
No custom backend	Project scope excludes a proxy server; API key is embedded in the client build
Response scope	Answers must stay within ReptiFiles leopard gecko caresheet content
Response length	Short enough to display in a single UI label during play (≤ 3 lines)
Concurrency	Only one request in flight at a time; player must not be able to double-send
Scene persistence	The API manager must survive scene transitions within the game
Async safety	HTTP response must not block the render loop or freeze the game

2.3 Available LLM APIs

Three API providers were considered for the backend model:

Provider	Relevant factors	Decision
Google Gemini	Free tier available, REST API well-documented, Flash Lite model is fast and low-cost	Selected
OpenAI GPT	Strong capability, but no free tier — cost prohibitive for a student research project	Rejected
Anthropic Claude	Strong instruction following, but similarly no free research tier available	Rejected

Gemini 3.1 Flash Lite was selected specifically because it offers low latency suitable for in-game use, a sufficiently free tier for the study's expected volume, and a straightforward REST endpoint that Godot's HTTPRequest node can call without any SDK dependency.

2.4 Integration approach options

Three approaches for integrating an LLM into Godot were evaluated:

Approach	Description	Assessment
Direct REST call from GDScript	GDScript makes an HTTP POST to the Gemini API endpoint using Godot's built-in HTTPRequest node	Selected — no external dependencies, works within engine
GDExtension / native plugin	A C++ or Rust plugin wraps an LLM SDK and exposes it to GDScript	Rejected — excessive complexity for the scope; SDK overhead not justified
Backend proxy server	A small Node/Python server relays prompts and responses; Godot calls localhost	Rejected — requires running a second process; out of project scope

The direct REST approach is the only one that requires no external infrastructure and fits entirely within Godot's built-in capabilities. The trade-off is that the API key is present in the exported build, which is acceptable for a controlled study environment where the build is not publicly distributed.

3. Design

3.1 Design principles

- **Separation of concerns.** All HTTP and JSON parsing logic is confined to a single script (GeminiManager.gd). All UI and prompt-building logic is confined to a separate script (ChatFunctionality.gd). Neither script calls the other's internal methods — they communicate only through Godot's signal system.
- **Singleton for shared API access.** GeminiManager is registered as an Autoload singleton so it persists across scene transitions and is accessible from any scene without a node reference. This avoids duplicating HTTP logic if the vet NPC appears in multiple scenes.
- **Asynchronous signal-based response handling.** HTTP requests in Godot are non-blocking by design. The GeminiManager emits a `response_received` signal when the API responds, and ChatFunctionality subscribes to this signal in `_ready()`. The game loop is never blocked.
- **Per-query HTTPRequest node lifecycle.** A new HTTPRequest node is instantiated for each outgoing query and freed immediately after response processing via `node.queue_free()`. This avoids state accumulation and keeps memory clean across an entire play session.
- **Prompt constraint wrapping.** Player input is never sent raw. Each message is prepended with an identity and task constraint string before dispatch. This keeps Erwin's persona and answer scope consistent without maintaining a stateful conversation history.

3.2 Component architecture

The system consists of two GDScript components inside the Godot runtime and one external service. Figure 1 shows the full component diagram.

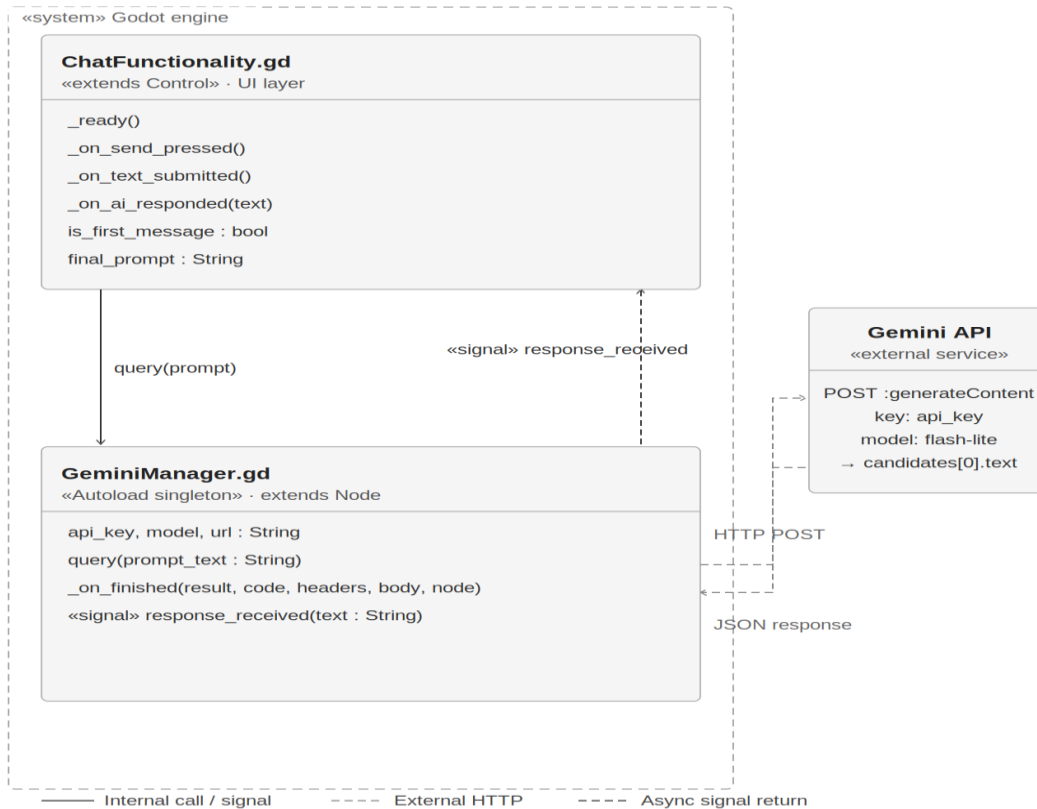


Figure 1. Component diagram of the LLM chatbot system. Solid arrows indicate internal calls and signal subscriptions; dashed arrows indicate external HTTP communication. The Godot boundary is shown as a dashed rectangle.

3.3 Component responsibilities

Component	Responsibility
ChatFunctionality.gd	Owns the chat UI. Handles player input events, constructs the final prompt (first-message intro or constrained follow-up), disables the send button during a pending request, and updates the ResponseLabel when the signal arrives.
GeminiManager.gd	Autoload singleton. Exposes a single public method query(). Creates an HTTPRequest node, serialises the prompt into the Gemini JSON body, fires the POST request, parses the response, emits response_received, and frees the HTTP node.
Gemini API	External REST service. Accepts a contents array with a parts/text field, returns a candidates array. The relevant text is at candidates[0].content.parts[0].text.

3.4 Data flow

The complete request-response cycle proceeds as follows:

- Player types a question and presses Send (or hits Enter).
- ChatFunctionality clears the input field, disables the send button, and sets ResponseLabel to 'Erwin is thinking...'.- ChatFunctionality checks is_first_message. On first call, it substitutes an introduction prompt; on subsequent calls it wraps the player's text in the identity/constraint string.
- ChatFunctionality calls GeminiManager.query(final_prompt).
- GeminiManager instantiates an HTTPRequest node, serialises the body as { contents: [{ parts: [{ text: prompt }]}]}, and fires POST to the Gemini endpoint.
- Gemini API processes the request and returns a JSON response.
- GeminiManager._on_finished() receives the response, navigates the nested JSON to candidates[0].content.parts[0].text, emits response_received(ai_text), and calls node.queue_free().
- ChatFunctionality._on_ai_responded() receives the signal, re-enables the send button, and sets ResponseLabel to the AI reply text.

3.5 Prompt engineering

Two distinct prompt patterns are used, selected by the is_first_message flag:

Introduction prompt (first message only)

Sent automatically when the player first opens the chat panel, before the player has typed anything. This prompt is not shown to the player:

```
You are Erwin Stevens, a virtual veterinarian in this reptile caretaking videogame. Introduce yourself warmly, explain that you are specialised in reptiles, and mention you are here to help. Keep it to 3 lines. Do not give caretaking tips yet.
```

Constrained follow-up prompt (all subsequent messages)

Each player question is wrapped in an identity/task/constraint triple before dispatch:

```
[Identity: Erwin Stevens, Virtual Vet.  
Task: Give reptile caretaking tips.  
Constraint: Exactly three lines of text.]  
Player asks: <player_input>
```

The constraint triple serves three purposes: it re-anchors the model's identity on every turn (preventing persona drift across the session), it restricts the topic domain to reptile caretaking, and it enforces a three-line length limit that fits the ResponseLabel without overflowing the UI. No conversation history is maintained between turns — each request is stateless — which is appropriate because the study only requires single-turn advisory responses rather than multi-turn dialogue.

3.6 Error handling

If the Gemini API returns a non-200 status code, or if the response body does not contain the expected candidates field, GeminiManager emits response_received with the string 'Error: Gemini API failed.' This ensures the ChatFunctionality always receives a signal and re-enables the send button, preventing the UI from becoming permanently disabled. The raw error code and body are printed to the Godot console for debugging.

4. Open issues and future work

- The API key is embedded in the client build. For a publicly distributed game this would require a proxy server; for the controlled study context it is acceptable.
- The three-line response constraint is enforced through the prompt, not through a post-processing step. Compliance depends on the model's instruction following. This should be monitored during the pilot run.
- No conversation history is maintained between turns. If multi-turn follow-up dialogue becomes a requirement in future iterations, a messages array will need to be accumulated in GeminiManager and passed with each request.
- Erwin's responses are not currently validated against the ReptiFiles caresheet programmatically. Content accuracy depends on the prompt constraint. A retrieval-augmented approach could improve fidelity in a future version.